



Administrator Linux. Professional

Nginx/Angie



Проверить, идет ли запись

Меня хорошо видно && слышно?



Преподаватель



Лавлинский Николай

Технический директор «Метод Лаб»

Более 15 лет в веб-разработке

Преподавал в ВУЗе более 10 лет
Более 5 лет в онлайн-образовании

Специализация: оптимизация производительности, ускорение сайтов и веб-приложений

https://t.me/methodlab_tg

<https://www.methodlab.ru/>

<https://rutube.ru/channel/24617406/>

<https://www.youtube.com/c/NickLavlinsky>

https://www.youtube.com/@site_support

<https://vkvideo.ru/@methodlab>

<https://habr.com/ru/users/Nickmob/>

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в Телеграм-чате



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом

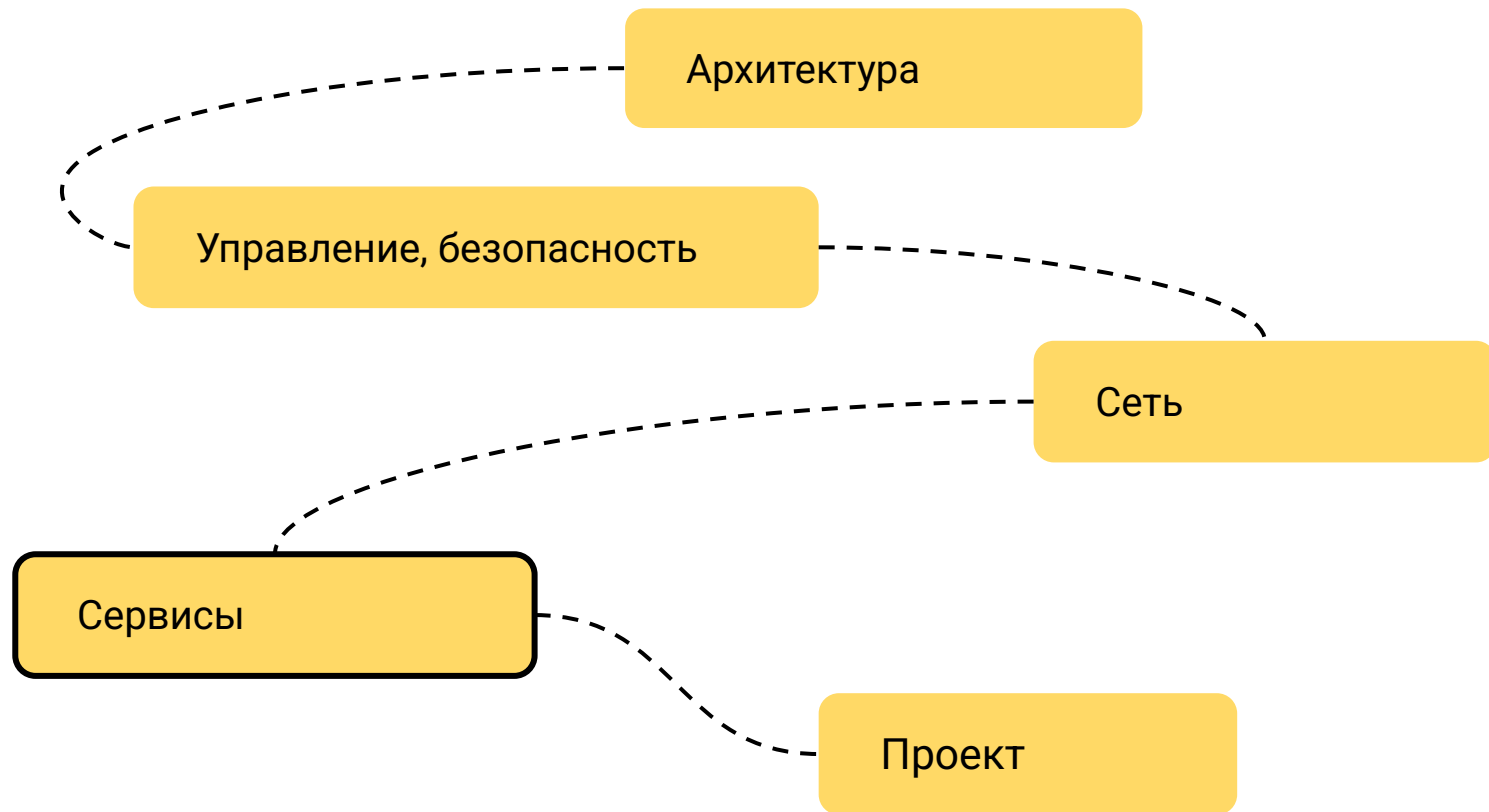


Документ

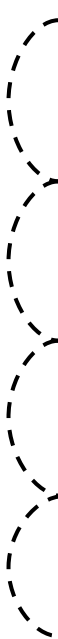


Ответьте себе или
задайте вопрос

Карта курса



Маршрут вебинара



Базовая конфигурация

Маршрутизация запросов

HTTP-проксирование

Перенаправления

Оптимизация доставки

Цели вебинара

После занятия вы сможете

1. Освоить базовые настройки Nginx/Angie
2. Понимать общую схему обработки запроса
3. Оптимизировать доставку контента

Смысл

Зачем вам это уметь

1. Администрировать веб-сервисы
2. Отлаживать проблемы с работой сайтов
3. Проксировать веб-приложения
4. Оптимизировать доставку контента



Базовая конфигурация Nginx/Angie

Nginx

- Один из самых популярных в мире
- Разработан Игорем Сысоевым (первые версии)
- Легковесный
- Высокопроизводительный
- Может работать как веб-сервер, прокси, балансировщик, кэш
- HTTP/1.1, HTTP/2, HTTP/3, TLS 1.3, Gzip, Brotli...
- Форк от создателей: Angie (<https://angie.software/>)



NGINX®
Part of F5



Angie

- Форк Nginx с отдельной командой
- Вся функциональность Nginx
- Встроенные средства мониторинга
- Консоль с веб-интерфейсом
- ACME-модуль (TLS сертификаты)
- Готовые бинарные пакеты в репозитории
- <https://angie.software/>



Отличия Angie

- Поддержка HTTP/3 при соединении с бэкендами
- Использование BPF для HTTP/3
- Режим привязки сессий (sticky), при котором все запросы в рамках одной сессии будут направляться на один и тот же проксируемый сервер
- Механизм плавного ввода проксируемого сервера в работу после сбоя с помощью опции `slow_start` директивы `server`
- Возможность создавать собственные метрики для мониторинга



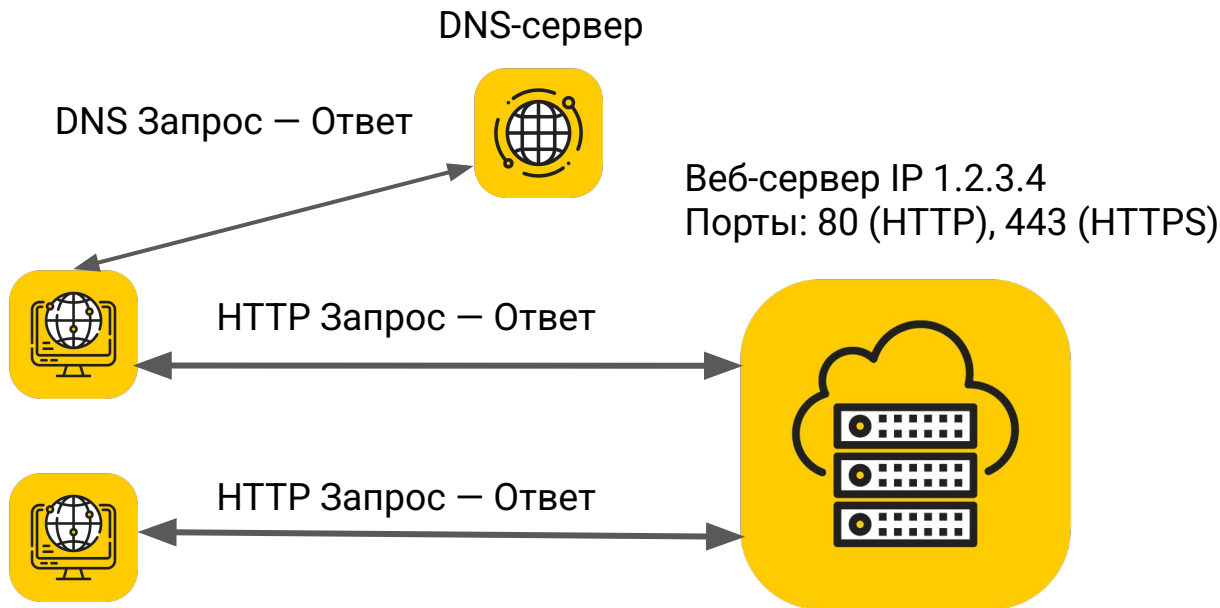
https://angie.software/http_upstream/

https://angie.software/stream_upstream/

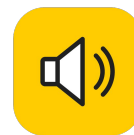
Nginx конфигурация

- Основной конфиг: `/etc/nginx/nginx.conf`
- Подключение дополнительных конфигов: `include /etc/nginx/conf.d/*.conf;`
- Отдельные сайты: блоки `server { }`
- Выбор сайта для обработки: директивы `listen` и `server_name`
- Конфигурация по URL: блоки `location { }`
- Синтаксис: директивы всегда заканчиваются `“;”`
- Директивы наследуются
- Некоторые блоки могут иметь вложенность
- Документация: <http://nginx.org/ru/docs/>

Базовая архитектура веб-приложения



Что, если сайтов на сервере несколько?



Маршрутизация запросов

Выбор сервера

- Сокет: `listen`
 - `listen 127.0.0.1:8000;`
 - `listen 127.0.0.1;`
 - `listen 80 default_server;`
 - `listen 8000; listen *:8000;`
 - `listen localhost:8000;`
 - `listen [::]:8000; listen [::1];`
 - `listen unix:/var/run/nginx.sock;`
- Доменное имя: `server_name`
 - `server_name test.ru www.test.ru;`
 - `server_name *.test.ru;`
 - `server_name ~^www\d+\.test\.ru$;`

Порядок поиска сервера по имени

1. Точное имя: `www.example.org`
2. Самое длинное * в начале: `"*.example.org"`
3. Самое длинное * в конце: `"api.*"`
4. Первое подходящее регулярное выражение (в порядке следования в конфигурационном файле)
5. Если нет совпадений: `default_server`

Несколько доменов в одном сервере

```
server {  
    server_name ~^(www\.)?(?<domain>.+)$; # www.site1.com site2.com  
    root /sites/$domain; # /sites/site1.com или /sites/site2.com  
}
```

- Выбор корня в зависимости от домена
- Осторожно с настройкой TLS-сертификатов

Поиск блока внутри сервера: location

- Обработка на базе URL
- URL приходит в обработанном виде
- [Нормализация](#)
- Преобразование путей
- Склейка повторных слешей (директива [merge_slashes](#))

Приоритеты видов location

1. `=` — точное совпадение (префиксный location)
 - `location = / { ... }`
2. `^~` — префиксный location без поиска regex
 - `location ^~ /images { ... }`
3. `~` или `~*` — регулярное выражение (с учётом и без учёта регистра)
 - `location ~* ^/. * { ... }`
4. префиксный location без модификатора
 - `location / { ... }`

Особенность обработки location

- Если location оканчивается на `/:` `location /uri/ { }`
- Используется `proxy_pass`
- Запрос без `/:` `/uri`
- Получаем редирект на `/uri/`
- Для обхода делаем 2 location
 - `location /uri/ { }`
 - `location = /uri { }`

Именованный location

- Определяется через @
- Используется только для перенаправлений
- Пример:

```
location / {  
    try_files $uri $uri/ @back;  
}  
location @back {  
    proxy_pass 127.0.0.1:8080;  
}
```

Примеры обработки proxy_pass

- location: /front без завершающего слеша
- proxy_pass: http://127.0.0.1:8000 без URI

query uri	result uri
http://127.0.0.1/front	/front
http://127.0.0.1/front/	/front/
http://127.0.0.1/frontotus	/frontotus
http://127.0.0.1/frontotus/	/frontotus/
http://127.0.0.1/front/otus	/front/otus
http://127.0.0.1/front/otus/	/front/otus/
http://127.0.0.1/front/otus/index.html	/front/otus/index.html

Примеры обработки proxy_pass

- location: /front без завершающего слеша
- proxy_pass: http://127.0.0.1:8000/ добавили URI /

query uri	result uri
http://127.0.0.1/front	/
http://127.0.0.1/front/	//
http://127.0.0.1/frontotus	/otus
http://127.0.0.1/frontotus/	/otus/
http://127.0.0.1/front/otus	//otus
http://127.0.0.1/front/otus/	//otus/
http://127.0.0.1/front/otus/index.html	//otus/index.html



Примеры обработки proxy_pass

- location: `/front/` с завершающим слешем
- proxy_pass: `http://127.0.0.1:8000/webapp` без завершающего слеша

query uri	result uri
<code>http://127.0.0.1/front</code>	301 <code>/front/</code>
<code>http://127.0.0.1/front/</code>	<code>/webapp</code>
<code>http://127.0.0.1/frontotus</code>	404
<code>http://127.0.0.1/frontotus/</code>	404
<code>http://127.0.0.1/front/otus</code>	<code>/webappotus</code>
<code>http://127.0.0.1/front/otus/</code>	<code>/webappotus/</code>
<code>http://127.0.0.1/front/otus/index.html</code>	<code>/webappotus/index.html</code>



Примеры обработки proxy_pass

- location: `/front/` с завершающим слешем
- proxy_pass: `http://127.0.0.1:8000/webapp/` с завершающим слешем

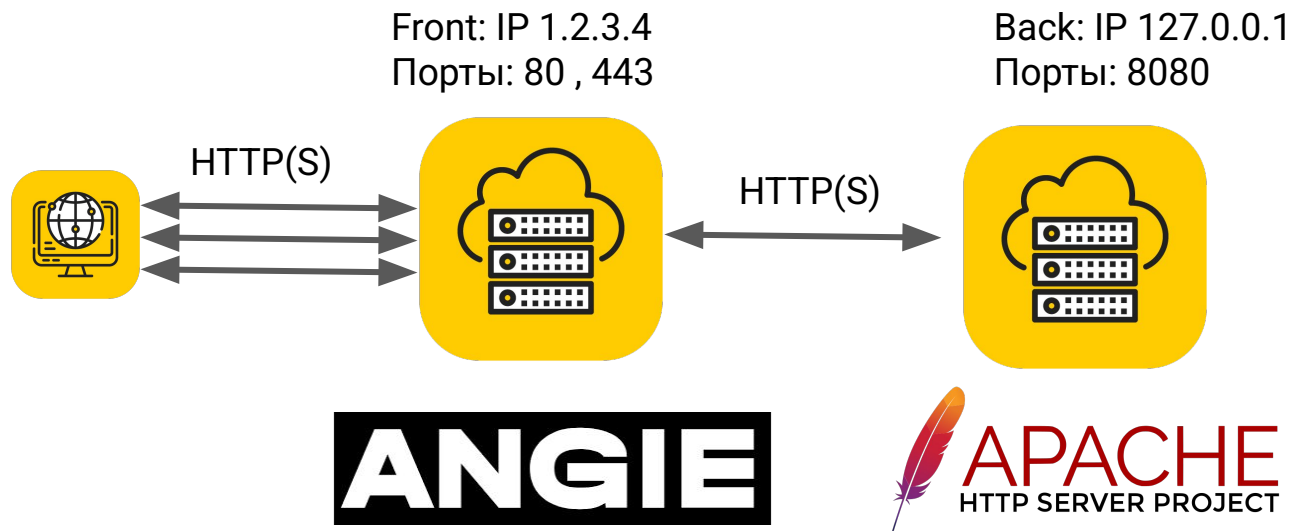
query uri	result uri
<code>http://127.0.0.1/front</code>	301 <code>/front/</code>
<code>http://127.0.0.1/front/</code>	<code>/webapp/</code>
<code>http://127.0.0.1/frontotus</code>	404
<code>http://127.0.0.1/frontotus/</code>	404
<code>http://127.0.0.1/front/otus</code>	<code>/webapp/otus</code>
<code>http://127.0.0.1/front/otus/</code>	<code>/webapp/otus/</code>
<code>http://127.0.0.1/front/otus/index.html</code>	<code>/webapp/otus/index.html</code>

Проксирование запросов

Варианты проксирования

- **proxy_pass** - HTTP proxy
- **fastcgi_pass** - FastCGI proxy (php)
- **uwsgi_pass** - UWSGI proxy (python)
- **scgi_pass** - SCGI proxy
- **memcached_pass** - memcached proxy (memcached)
- **grpc_pass** - GRPC proxy (grpc)

Front-end и back-end



- TLS
- Отдача статики
- Проксирование на Apache

- Динамика
- mod_php, mod_perl, mod_fcgi, mod_uwsgi

Настройка проксирования HTTP

```
location / {  
    proxy_pass http://localhost:8000/uri/;  
    proxy_pass http://unix:/tmp/backend.socket:/uri/;  
    proxy_pass http://upstream;  
    proxy_set_header Host $host;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    proxy_set_header X-Real-IP $remote_addr;  
}
```

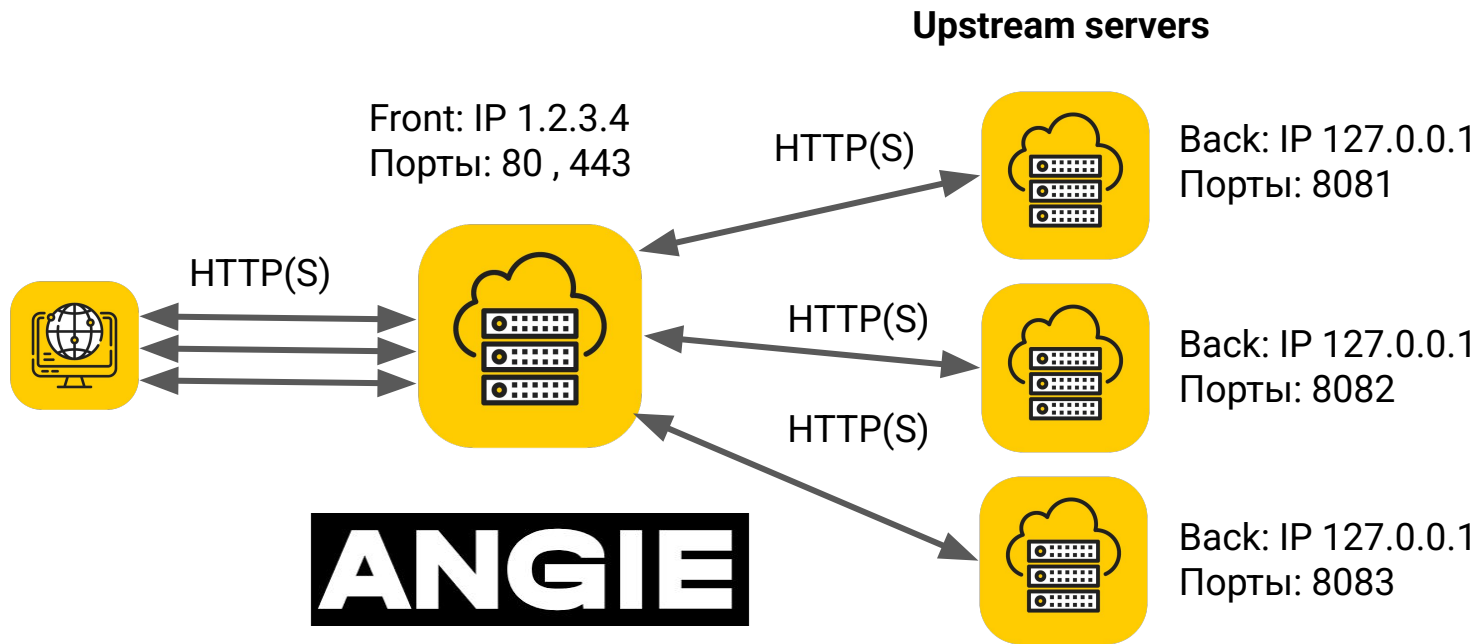
Настройка балансировки (upstream)

```
upstream backend {  
    server backend1.example.com weight=5;  
    server backend2.example.com:8080;  
    server unix:/tmp/backend3;  
    server backup1.example.com:8080 backup;  
    server backup2.example.com:8080 backup;  
}  
  
server {  
    location / {  
        proxy_pass http://backend;  
    }  
}
```

Параметры директивы server

- `weight` – вес сервера
- `max_conns` – максимальное количество одновременных подключений
- `backup` – запасной сервер
- `down` – недоступный сервер
- `max_fails` – максимальное количество ошибок
- `fail_timeout` – время для определения недоступности сервера (`max_fails`)

Балансировка с Nginx



Вопросы?



Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет



Перенаправления (редиректы)

Директива return

- Возврат кода ответа
- Перенаправление (301, 302)
- Примеры:
 - `return (301 | 302 | 303 | 307) url;`
 - `return (1xx | 2xx | 4xx | 5xx) ["text"];`

Использование return

- На другой домен:
 - `server_name www.old.com old.com;`
 - `return 301 $scheme://www.new.com$request_uri;`
- Исправление домена:
 - `server {`
 - `listen 80 default_server;`
 - `listen 443 ssl default_server;`
 - `server_name _;`
 - `return 301 $scheme://www.example.com;`
 - `}`
- Перенаправление на HTTPS:
 - `return 301 https://example.com$request_uri;`

Директива rewrite

- Перенаправление на другой URL
- Синтаксис: `rewrite regex URL [flag];`
- Флаги
 - `last` – завершаем обработку `rewrite`, ищем новый `location`
 - `break` – завершаем обработку `rewrite`, остаёмся в текущем `location`
 - `redirect` – возвращаем клиенту 302
 - `permanent` – возвращаем клиенту 301

Использование rewrite

- Обычный rewrite:

```
rewrite ^/result/([a-zA-Z0-9_]+)$ /result/$1/ permanent;
```

- По GET-параметрам (/index?uid=425&doc=18 -> /users/425/documents/18)

```
location /index {  
    if ($args ~ "^uid=(\d+)&doc=(\d+)") {  
        set $key_uid $1;  
        set $key_doc $2;  
        rewrite ^.*$ /users/$key_uid/documents/$key_doc? last;  
    }  
}
```

- Еще вариант (/index?uid=425&doc=18 -> /users/425/documents/18):

```
location /index {  
    proxy_pass http://127.0.0.1:8080/users/$arg_uid/documents/$arg_doc;  
}
```

Директива try_files

- Вариант последовательного перебора путей обработки запроса
- Пример:

```
location / {  
    try_files /system/maintenance.html $uri $uri/index.html $uri.html @namedloc;  
}  
  
location @namedloc {  
    proxy_pass http://127.0.0.1:8080;  
}
```

Оптимизация доставки

Настройка gzip

```
gzip on;  
gzip_static on;  
gzip_types text/plain text/css text/xml application/javascript  
application/json application/msword application/rtf application/pdf  
application/vnd.ms-excel image/x-icon image/svg+xml application/font-ttf;  
gzip_comp_level 3;  
gzip_proxied any;  
gzip_min_length 1000;  
gzip_disable "msie6";  
gzip_vary on;
```

Настройка brotli

```
brotli            on;  
brotli_static     on;  
brotli_comp_level 3;  
brotli_types      text/plain text/xml text/css application/javascript  
application/json image/x-icon image/svg+xml;
```

https://github.com/google/ngx_brotli



Настройка zstd

```
zstd                on;  
zstd_min_length     256;  
zstd_comp_level     3;  
zstd_static         on;  
zstd_types          text/plain text/css text/xml application/javascript  
application/json image/x-icon image/svg+xml;
```

<https://github.com/tokers/zstd-nginx-module>
<https://www.youtube.com/watch?v=ILThGyFPZIU>

Заголовки кэширования

```
location /static {  
    add_header Cache-Control "max-age=31536000, public, no-transform, immutable";  
}
```



Серверное кэширование

nginx.conf:

```
proxy_cache_valid 1m;  
proxy_cache_key $scheme$host$request_uri;  
proxy_cache_path /cache levels=1:2 keys_zone=one:10m inactive=48h max_size=800m;
```

Обязательно включить буферизацию ответа: proxy_buffering on

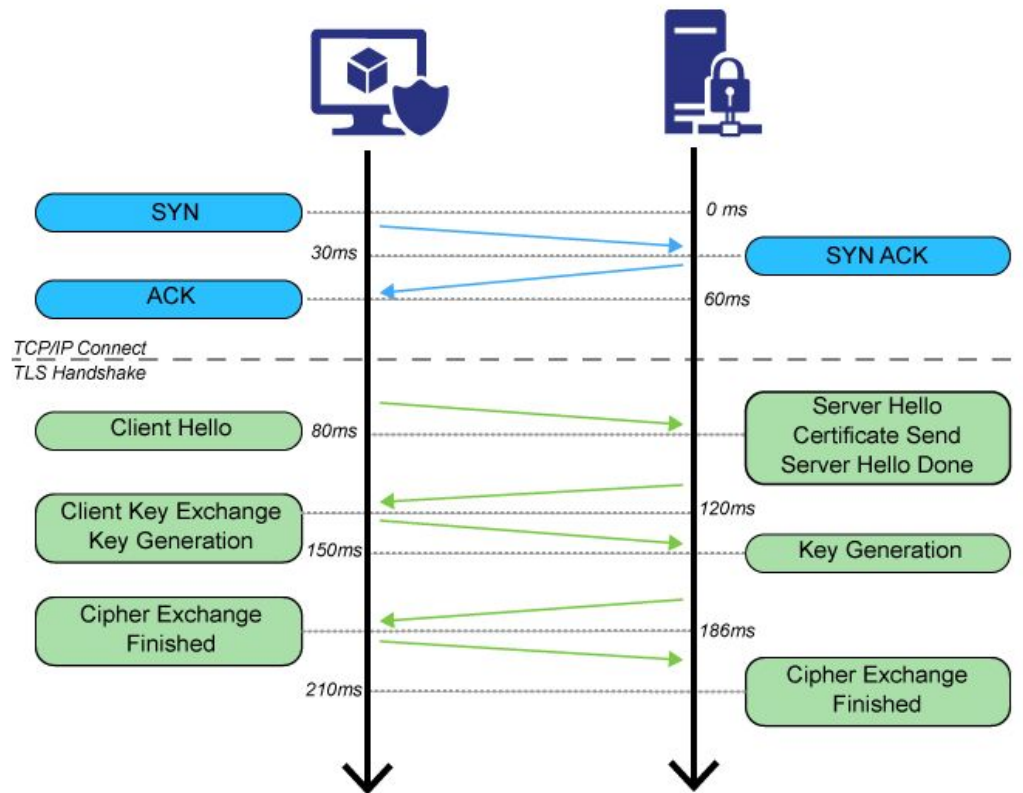
server.conf:

```
location / {  
    proxy_cache one;  
    proxy_cache_valid 200 1h;  
    proxy_cache_lock on;  
    proxy_cache_use_stale updating error timeout invalid_header http_500 http_502 http_504;  
    proxy_cache_background_update on;  
}
```

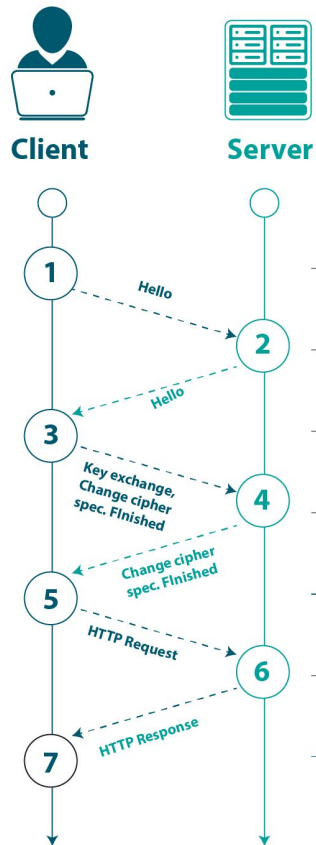
https://angie.software/angie/docs/configuration/modules/http/http_proxy/#proxy-cache
<https://habr.com/ru/articles/929634/>



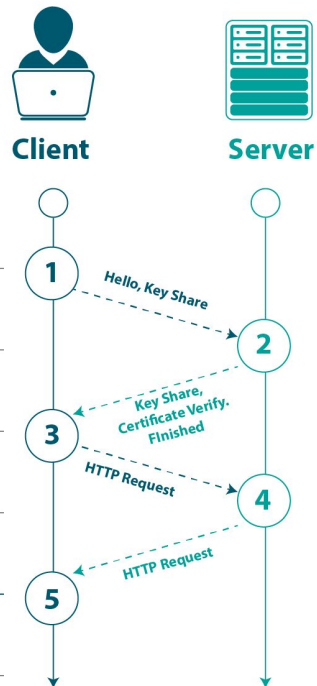
Настройка TLS



TLS 1.2 (Full Handshake)



TLS 1.3 (Full Handshake)



Настройка TLS

```
ssl_prefer_server_ciphers    off;
ssl_protocols                TLSv1.2 TLSv1.3;
ssl_ciphers
ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-S
HA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20
-POLY1305:DHE-RSA-AES128-GCM-SHA256:DHE-RSA-AES256-GCM-SHA384:DHE-RSA-CHACHA20-POL
Y1305;

ssl_early_data              on;
ssl_session_tickets         on;
ssl_session_timeout         28h;
ssl_session_cache           shared:SSL:10m;
```

Настройка HTTP/2

- Достаточно настроить HTTPS добавить директиву включения HTTP/2
- Помним о требовании версии OpenSSL ($\geq 1.0.2$), согласование за счет ALPN
- Конфигурация:

```
http2 on;
```

- Дополнительные параметры:

```
http2_max_concurrent_streams 128;
```

```
http2_chunk_size 8k;
```

Настройка HTTP/3

- Добавляем новый сокет (UDP 443)

```
listen 443 quic reuseport;
```

- Конфигурация (включение и заголовок для перехода на H3):

```
http3 on;
```

```
add_header Alt-Svc 'h3=":443"; ma=86400';
```

- Дополнительные параметры:

```
quic_gso on;
```

```
quic_bpf on;
```

```
quic_retry off;
```

```
http3_max_concurrent_streams 128;
```

Вопросы?



Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет

Что мы изучили?

Подведем итоги

1. Базовые настройки Nginx
2. Маршрутизацию запросов
3. Перенаправления
4. Оптимизацию доставки



Список материалов для изучения

1. <http://nginx.org/ru/docs/>
2. <https://nginx.org/en/docs/varindex.html>
3. <https://blog.debian-help.ru/2014/12/optimizaciya-nginx-parametri-sendfile-tcpnodelay-tcpnopush>
4. <https://blog.nginx.org/>
5. <https://www.youtube.com/user/NginxInc>
6. https://www.youtube.com/channel/UCf1P_nESQ6hPITwkH5Vik1A
7. https://www.youtube.com/channel/UCH_PvxSiuETI_ecbeAmb7_w

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет



Тест

Вопросы для проверки

Каким символом должны всегда заканчиваться директивы в конфигурационном файле Nginx?

- A. Двоеточием (:).
- B. Фигурной скобкой ({}).
- C. Точкой с запятой (;).
- D. Двойным двоеточием (::).



Отправьте в чат верную букву

Вопросы для проверки

Каким символом должны всегда заканчиваться директивы в конфигурационном файле Nginx?

- A. Двоеточием (:).
- B. Фигурной скобкой ({}).
- C. Точкой с запятой (;).**
- D. Двойным двоеточием (::).

Вопросы для проверки

Согласно порядку поиска сервера по имени в Nginx, какой тип совпадения имеет наивысший приоритет при выборе блока `server {}`?

- A. Самое длинное совпадение с регулярным выражением (~).
- B. Самое длинное совпадение с маской в конце (api.*).
- C. Самое длинное совпадение с маской в начале (*.example.org).
- D. Точное имя (www.example.org).



Отправьте в чат верную букву



Вопросы для проверки

Согласно порядку поиска сервера по имени в Nginx, какой тип совпадения имеет наивысший приоритет при выборе блока `server {}`?

- A. Самое длинное совпадение с регулярным выражением (~).
- B. Самое длинное совпадение с маской в конце (api.*).
- C. Самое длинное совпадение с маской в начале (*.example.org).
- D. Точное имя (www.example.org).



Вопросы для проверки

Какой модификатор location имеет наивысший приоритет и используется для точного совпадения с URI, останавливая дальнейший поиск совпадений?

- A. Точное совпадение (=).
- B. Модификатор приоритета (^~).
- C. Регулярное выражение (~*).
- D. Префиксный location без модификатора.



Отправьте в чат верную букву

Вопросы для проверки

Какой модификатор location имеет наивысший приоритет и используется для точного совпадения с URI, останавливая дальнейший поиск совпадений?

A. Точное совпадение (=).

B. Модификатор приоритета (^~).

C. Регулярное выражение (~*).

D. Префиксный location без модификатора.

Вопросы для проверки

Какая директива используется в Nginx для проксирования запросов на сервер, работающий по протоколу FastCGI (часто используется для PHP-приложений)?

- A. proxy_pass.
- B. scgi_pass.
- C. fastcgi_pass.
- D. uwsgi_pass.



Отправьте в чат верную букву



Вопросы для проверки

Какая директива используется в Nginx для проксирования запросов на сервер, работающий по протоколу FastCGI (часто используется для PHP-приложений)?

- A. proxy_pass.
- B. scgi_pass.
- C. fastcgi_pass.**
- D. uwsgi_pass.

Вопросы для проверки

Какая директива используется в глобальной части конфигурации Nginx (nginx.conf) для определения пути, зоны памяти и параметров серверного кэша?

- A. add_header Cache-Control.
- B. proxy_cache_path.
- C. proxy_cache_valid.
- D. proxy_cache_key.



Отправьте в чат верную букву



Вопросы для проверки

Какая директива используется в глобальной части конфигурации Nginx (nginx.conf) для определения пути, зоны памяти и параметров серверного кэша?

A. add_header Cache-Control.

B. proxy_cache_path.

C. proxy_cache_valid.

D. proxy_cache_key.

Рефлексия

Цели вебинара

Проверка достижения целей

1. Освоили базовые настройки Nginx
2. Разобрали общую схему обработки запроса
3. Оптимизировали доставку контента



Рефлексия



Что было самым полезным на занятии?



Как будете применять на практике то, что узнали на вебинаре?

**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

Спасибо за внимание!

Приходите на следующие вебинары



Лавлинский Николай

Технический директор “Метод Лаб”

https://t.me/methodlab_tg
<https://www.methodlab.ru/>
<https://rutube.ru/channel/24617406/>
<https://www.youtube.com/c/NickLavlinsky>
https://www.youtube.com/@site_support
<https://vkvideo.ru/@methodlab>
<https://habr.com/ru/users/Nickmob/>

